

LLMOPS LIFECYCLE, PROMPT VERSIONING & AI EVALUATION

Study Guide — Hardening Production AIP Functions and Chatbots

Week 6 · Day 1 · Prepared by Rajesh Pasham

Program Context

Item	Detail
Session Type	Concepts and architecture lecture — no live platform work today
Audience	India and Europe batches together — this session is common to all four batches
This Week's Focus	Production AIP, LLMOps & Governance — hardening the Copilots, chatbots, and agent workflows built in Weeks 2–5
This Week's Deliverable (Day 3)	A Production Readiness Package: an evaluation suite, an observability dashboard, and a release plan

Where Today Fits

Weeks 2 through 5 built things that work — a Claims Copilot that answers questions, chatbots and functions that write back to the Ontology, and agents that plan and act on their own. None of that was tested the way a production system needs to be. Week 6 is where you stop asking “does it work in the demo?” and start asking “how do we know it still works, every day, for every policyholder, and how would we know the moment it doesn't?”

Today's four topics — the LLMOps lifecycle, prompt versioning, model evaluation, and observability — are the first half of that production-readiness bar. Tomorrow's session covers the second half: drift monitoring, cost management, release strategy, and governance. Concretely, your Batch 1 ClaimsCopilot, Batch 2 UnderwritingCopilot, Batch 3 FraudTriageCopilot, Batch 4 ServicingCopilot, and the FraudRiskAssessment, RiskScoreRefresh, ComplianceReadinessCheck, and CoverageRecommendationAssessment functions and chatbots you built in Week 5 are exactly the systems today's four topics apply to — keep yours in mind as you work through this guide.

Takeaway

Nothing you built in Weeks 2–5 is being replaced. Today adds a discipline around it: version it, evaluate it, and watch it — the same system, held to a higher bar.

Learning Objectives

By the end of today, you will be able to:

- Explain the LLMOps lifecycle and how it differs from classical MLOps
- Explain why prompts require version-control discipline, tied to Week 5's system/task prompts
- Describe the core concepts behind AIP Evals: Evaluation Suite, Target Function, Evaluation Function, Test Cases, and Metrics
- Apply those concepts to a function or chatbot you've already built
- Describe what Monitoring, Debugging, and Analysis each cover in Foundry Observability
- Preview Day 2 and the Week 6 Production Readiness Package

Topic 1 — The LLMOps Lifecycle

1. From MLOps to LLMOps

- **Classical MLOps assumes a deterministic, numeric model** — a fraud-scoring model trained on structured data gives the same score for the same input, every time. Your own RiskScoreRefresh function (Batch 2) has exactly this kind of piece inside it — the riskScoreStale check is a plain deterministic calculation, which is why it was never built as a Use LLM block.
- **An LLM-powered function does not give that guarantee** — the same prompt, sent to the same model, can return a differently worded, or differently reasoned, response on two separate calls. Feed your Batch 1 ClaimsCopilot the same claim narrative twice and its summary wording can legitimately differ both times — that is the non-determinism this week's evaluation discipline exists to manage.
- **The unit of operation changes too** — classical MLOps versions a trained model artifact; LLMOps versions a model choice, a prompt, and the tools a function or chatbot can call. Your Batch 3 Compliance Chatbot makes this concrete: its model choice, its system prompt, and its access to the ComplianceReadinessCheck and SubmitComplianceReview tools are three separate things that each need to be tracked, not one.

2. What the LLMOps Lifecycle Covers

A continuous loop, not a one-time deployment checklist:

- **Build** — the AIP Logic functions and AIP Chatbot Studio chatbots you've been building since Week 2 — prompts, tool access, and model selection, authored and iterated. Your Batch 1 FraudRiskAssessment function and Fraud Detection Chatbot are this stage in practice: a Use LLM fraud-judgment block wired to an OpenInvestigation tool.
- **Evaluate** — testing behavior against defined test cases before and after any change, through AIP Evals (covered in depth below). The Evaluation Suite requirement on your Batch 2 RiskScoreRefresh function — 3+ Test Cases before it counts as done — is this stage, not theory.
- **Observe & Operate** — monitoring behavior once a function or chatbot is in live use, catching regressions and failures quickly. Opening your Batch 3 Compliance Chatbot's own Monitoring tab to confirm it actually called ComplianceReadinessCheck is a small, real example — today's Observability topic, with drift and cost covered tomorrow.
- **Govern & Release** — controlled rollout of changes, audit trails, and responsible-AI guardrails. The confirmation-required setting on your Batch 4 EscalateForCoverageRecommendation Action — a person must approve it before it writes to the Ontology — is a governance control you already built, not a future one. Day 2's focus.

3. Classical MLOps vs. LLMOps

Dimension	Classical MLOps	LLMOps
What's versioned	A trained model artifact	A model choice, a prompt, and the tools it can call
Determinism	Same input, same output, reliably	Same input can produce different output — expected
Testing cadence	Validate once before release	Continuous evaluation, after every prompt or model change
What “drift” means	The input data distribution shifts over time	Output quality or behavior can shift with no visible input change

Dimension	Classical MLOps	LLMOps
Primary new artifact	Training data and model weights	The prompt — now reviewed, versioned, and tested like code

In your own systems: your Batch 4 CoverageRecommendationAssessment function bundles a model choice, a judgment prompt, and tool access as the one thing now versioned together; your Batch 1 ClaimsCopilot can legitimately phrase the same claim summary two different ways on two runs; a Test Case for FraudRiskAssessment (Batch 1) — claimAmount = 48,000, isHighValueClaim = true, expected recommendedFraudAction = OpenInvestigation — needs re-running after every prompt or model change, not just once before go-live; and your Batch 2 RiskScoreRefresh function's judgment can shift after a platform model update even though riskScoreStale's logic never changed — that is LLM drift.

4. Why Continuous Evaluation Replaces One-Time Testing

- **The model underneath can change** — a platform-level model update can shift how a prompt is interpreted, even when nothing in your function or chatbot changed at all — including the Use LLM risk-judgment block inside your Batch 2 RiskScoreRefresh function, whose deterministic riskScoreStale calculation would stay identical while its judgment output quietly shifts.
- **The prompt itself changes over time** — a system or task prompt refined after user feedback is a new artifact; it needs the same evaluation the original version had, not an assumption that “it still works.” The instructions inside your Batch 3 Compliance Chatbot's system prompt are exactly this kind of artifact once someone edits them.
- **Non-determinism means one passing run isn't proof** — because the same input can produce different outputs across calls, a single successful test run is weaker evidence than it would be for a deterministic model. This is why the Evaluation Suite built for your Batch 1 FraudRiskAssessment function is required to carry 3+ Test Cases and get run repeatedly — one pass proves far less than it would for a classical model.

Common Mistake

A working demo is not evidence of production readiness. LLM non-determinism means a system that performed well in ten test conversations can still fail on the eleventh in a way none of those ten runs revealed.

Topic 2 — Prompt Versioning

5. The Prompt Is a First-Class Artifact

- **Recall Week 5's system and task prompts** — every chatbot built in AIP Chatbot Studio, and every AIP Logic function using a Use LLM block, is shaped as much by its prompt as by the model behind it. Your Batch 2 Underwriting Chatbot's system prompt — describing its persona, scope, and the rule to call UnderwritingDecisionReasoning before InvokeRiskAssessmentChatbot — is doing as much work as the model choice underneath it.
- **A prompt change is a behavior change** — editing a task prompt's wording, tone, or instructions changes what the function or chatbot does. Loosen the instruction inside your Batch 3 Fraud Detection Chatbot's system prompt that it “never states a fraud read without having called [its] tool,” and the chatbot could start guessing — the same category of change a code edit would be, and it deserves the same scrutiny.
- **Without versioning, a regression is invisible until a user notices** — if nobody recorded what the prompt said last week, there is no way to confirm whether this week's change caused a complaint, or to revert to the version that worked. A silent edit to your Batch 4 Customer Support Chatbot's escalation rule, with no record of the prior wording, is exactly this risk.

6. What Good Prompt Versioning Looks Like

- **Every meaningful edit is a new, labeled version** — not a silent overwrite — a record of what changed, when, and by whom, the same way a commit records a code change. This is the discipline behind Publishing FraudRiskAssessment's (Batch 1) final version only after a deliberate change, rather than overwriting it mid-edit.
- **A version is tied to the evaluation results it produced** — each prompt version should be traceable to the AIP Evals run that validated it, so “this is the version we approved” is a verifiable claim, not a memory. The 3+ Test Cases required for CoverageRecommendationAssessment (Batch 4) are what that specific published version was actually checked against.
- **Rollback is a real, exercised option** — if a new prompt version underperforms once live, reverting to the last known-good version should be fast and unambiguous. If a revised judgment prompt inside your Batch 2 RiskScoreRefresh function started mis-scoring stale risk reads, a recorded prior version makes reverting a two-minute fix instead of a reconstruction project.

7. Prompt Versioning in Practice: A Chatbot Example

Tracing one prompt change through your Batch 4 Customer Support Chatbot, from idea to validated release:

1. The Customer Support Chatbot's task prompt is edited to handle a new retention-offer scenario

The change — tightening when it should hand off to the Policy Recommendation Chatbot via EscalateForCoverageRecommendation — is written up as a new version, not applied silently over the previous wording.

2. The new version is run through its Evaluation Suite before anything else happens

The same test cases used to validate the prior version, plus new ones covering the retention-offer handoff, including a case like POL268's Renewal REN031.

3. Metrics from the new version are compared against the prior version's

Not just “does it pass” — whether accuracy and tone metrics hold steady or improve on the existing test cases, and whether the handoff to CoverageRecommendationAssessment still fires at the right moments.

4. Only once Metrics are acceptable does the new version replace the old one in production

The prior version stays recorded and retrievable, so a regression discovered later can be rolled back to a known point.

Checkpoint — Confirm Before Continuing

Before moving on: can you name the last time you edited a system or task prompt in one of your own chatbots or functions (your Underwriting Chatbot, Fraud Detection Chatbot, Compliance Chatbot, or Customer Support Chatbot, for instance)? Could you reconstruct exactly what it said beforehand?

Topic 3 — Model Evaluation with AIP Evals

8. Why AIP Evals Exists

- **A conventional unit test assumes one correct output** — an LLM-powered function may produce several differently worded but equally valid outputs — two runs of your Batch 1 ClaimsCopilot against the same claim narrative can both be correct while using different phrasing. AIP Evals is built to score that kind of result, not just an exact string match.
- **It evaluates AIP Logic functions, AIP Chatbot Studio chatbots, and code-authored functions** — any of the function types you've built across this program can be the subject of an evaluation — the same mechanism applies to your Batch 1 FraudRiskAssessment function and to your Batch 1 Fraud Detection Chatbot that calls it.

- **It integrates with AI FDE for conversational creation of evaluations** — an Evaluation Suite can be scaffolded through a conversational interface rather than built field by field — the same time saving as using Add as test case straight from a Preview run of ComplianceReadinessCheck (Batch 3), rather than typing each Test Case by hand.

9. The Five Core Concepts of AIP Evals

- **Evaluation Suite** — the overarching collection that ties everything together — it combines a set of Test Cases, one or more Target Functions, and the Evaluation Function(s) used to judge them. The Evaluation Suite built for your Batch 1 FraudRiskAssessment function is exactly this: its 3+ Test Cases, FraudRiskAssessment as the Target Function, and an Evaluation Function comparing its fraudRiskTier output to what was expected.
- **Target Function** — the function actually being evaluated: an AIP Logic function, a chatbot function, or a code-authored function. Your Batch 2 RiskScoreRefresh function is a Target Function; so, separately, could be the Risk Assessment Chatbot (Batch 2) that calls it, since a single suite can evaluate more than one Target Function at once.
- **Evaluation Function** — the method used to compare a Target Function's actual output against the expected output defined in a test case. For ComplianceReadinessCheck (Batch 3), that means checking whether its regulatorySignOffRequired and complianceReadinessTier outputs match what the test case expected — the scoring logic behind the Suite.
- **Test Cases** — defined sets of inputs and expected outputs, passed into the Evaluation Function(s) each time a Suite runs. A concrete Test Case for FraudRiskAssessment (Batch 1): input claimAmount = 48,000 and isHighValueClaim = true, expected output recommendedFraudAction = OpenInvestigation.
- **Metrics** — the quantifiable, per-test-case results a Suite run produces — enabling both aggregate scoring across all test cases and comparison across multiple runs. Running the CoverageRecommendationAssessment (Batch 4) Suite five times and watching whether its pass rate on the EscalateForCoverageRecommendation test cases stays stable is exactly this kind of comparison.

Platform Mechanic

An Evaluation Suite run passes each Test Case's input into the Target Function, hands the Target Function's actual output and the Test Case's expected output to the Evaluation Function, and records the result as a Metric. Walked through concretely: a Test Case with claimAmount = 48,000 and isHighValueClaim = true goes into your Batch 1 FraudRiskAssessment Target Function; its actual recommendedFraudAction output is compared by the Evaluation Function against the expected OpenInvestigation; the match or mismatch becomes a Metric. Because of LLM non-determinism, the value of AIP Evals comes from running this cycle repeatedly and comparing Metrics across runs — not from a single pass.

10. Worked Example: Evaluating FraudRiskAssessment (Batch 1)

- **Target Function** — your Batch 1 FraudRiskAssessment function — it takes a claim's claimAmount and isHighValueClaim and recommends a fraudRiskTier and recommendedFraudAction, such as OpenInvestigation.
- **Test Cases** — representative claims with a known, agreed-upon expected recommendation each — for example, claimAmount = 48,000 and isHighValueClaim = true, expecting recommendedFraudAction = OpenInvestigation, plus edge cases near the 25,000 High-Value threshold.
- **Evaluation Function** — a comparison method judging whether FraudRiskAssessment's recommendedFraudAction and fraudRiskTier match what each Test Case expected, in substance, not necessarily in exact wording.

- **Metrics, run one** — the Suite runs once against all Test Cases, producing a per-test-case pass/fail — did recommendedFraudAction match expected? — plus an aggregate pass rate across the whole set.
- **Metrics, compared across runs** — the same Suite is run again — because of non-determinism, a single run's aggregate score is not proof of reliability. Comparing pass rates across several runs of the 48,000-claim Test Case is what tells you whether FraudRiskAssessment is consistently correct or only correct by chance.
- **The decision this supports** — only once Metrics are both acceptable in aggregate and stable across repeated runs does this become evidence FraudRiskAssessment is ready for a production release — the same bar any prompt version or model update should be held to going forward.

Topic 4 — AI Observability

11. Why Observability Is a Separate Concern from Evaluation

- **Evaluation is proactive, run against known test cases** — AIP Evals tells you how FraudRiskAssessment (Batch 1) performs on the claimAmount = 48,000 Test Case you defined in advance; it cannot tell you what real policyholders are actually asking the Claims Chatbot, or how the function behaves on a claim pattern nobody anticipated.
- **Observability is live, running against real traffic** — Foundry Observability covers what is happening right now, across real executions — opening your Batch 3 Compliance Chatbot's Monitoring tab to see its actual conversations with real users, not test cases, is this in practice.
- **A production-ready system needs both** — passing Metrics from your Batch 2 RiskScoreRefresh Evaluation Suite and a clean observability picture for the live Risk Assessment Chatbot answer different questions; neither one substitutes for the other.

12. Foundry Observability: Monitoring

- **Data Health and rule-based monitoring** — conditions defined against your Functions, Actions, and AIP Logic executions, checked continuously. A rule against your Batch 1 FraudRiskAssessment function — alert if its failure rate crosses a set threshold — is checked continuously rather than only when someone happens to look.
- **Alerts reach the team where it already works** — rule violations can notify through PagerDuty, Slack, webhooks, or Foundry's own notifications. A spike in failed calls to InvokeFraudDetectionChatbot (Batch 1) reaching your team's Slack channel the moment it happens is the practical form of this.
- **Thirty days of performance data** — enough history to spot a trend — a slow increase in failure rate, or P95 execution duration creeping upward for your Batch 2 RiskScoreRefresh function — not just a single bad moment.

13. Foundry Observability: Debugging

- **Workflow Lineage and execution history** — seven days of execution history, letting you trace a specific run of a Function, Action, or AIP Logic chain back through what it actually did — for example, following one SubmitComplianceReview (Batch 3) submission back through ComplianceReadinessCheck to the Fraud Case that triggered it.
- **Log search across executions** — searching logs across many executions at once, rather than opening one run at a time — searching every InvokePolicyRecommendationChatbot (Batch 4) call from the past week for a shared error signature, when you're looking for a pattern rather than a single incident.
- **Tracing** — the ability to drill into any single operation within an execution to see its duration, its inputs, its outputs, and any errors. Tracing into one failed run of FraudRiskAssessment (Batch 1)

to see the exact claimAmount it received and where in the chain it errored is the level of detail that turns “it failed” into “here is exactly where, and why.”

14. Foundry Observability: Analysis

- **Export logs, metrics, and traces to a streaming dataset** — observability data isn't locked inside a fixed view — a month of your Batch 3 Compliance Chatbot's traces can be exported and analyzed for a slow shift in how often it recommends escalation, a question no single built-in view answers directly.
- **Confirmed tracked metrics** — success and failure counts, P95 execution duration, and execution counts across resources — tracked for your Batch 2 RiskScoreRefresh function and the Risk Assessment Chatbot calling it, the same way as for any other Function, Action, or AIP Logic chain.
- **Where LLM-specific tracking goes further** — token usage, cost per call, and output-quality drift are LLM Ops practices your team should establish deliberately, not platform features to assume exist out of the box. This mirrors a well-known pattern in production LLM deployments more broadly: teams commonly track token volume and per-call cost in their own dashboard, the same way they'd track any other cost that can grow quietly with traffic. Day 2 returns to cost management directly.

Takeaway

Monitoring, Debugging, and Analysis answer three different questions: is anything wrong right now, what exactly happened on this one run, and what's the pattern across many runs. A production-ready system needs all three, not just one.

15. Looking Ahead: Day 2 and the Week 6 Deliverable

- **Day 2 — Production, Cost & Governance** — drift monitoring, cost management, release strategy, capacity planning, AI governance and responsible AI, and audit and compliance — the second half of what production-readiness requires.
- **Day 3 — Self-paced build** — you will build an evaluation suite, an observability dashboard, and a release plan for one of your program's existing functions or chatbots — today's four topics, applied hands-on.
- **Day 4 and 5 — Batch practicals and review** — working sessions and review of the Production Readiness Package each batch produces, closing out Week 6.

16. Common Pitfalls

- **Treating a working demo as evidence the system is production-ready** — your Batch 1 ClaimsCopilot answering well in ten test conversations has not been evaluated; LLM non-determinism means it can fail on the eleventh in a way none of your ten runs revealed.
- **Editing a system prompt directly in production with no history** — a prompt is as central to behavior as code. Editing your Batch 4 Customer Support Chatbot's escalation rule without a version history makes it impossible to know what changed, or to roll back when a change goes wrong.
- **Running an evaluation once, before launch, and never again** — an Evaluation Suite run once at launch for RiskScoreRefresh (Batch 2) tells you nothing about a later model update, a prompt edit six weeks on, or a slow drift in output quality; evaluation is continuous, not a one-time gate.
- **Treating Metrics from a single run as conclusive** — LLM non-determinism means one run's Metrics for ComplianceReadinessCheck (Batch 3) can look fine by chance; AIP Evals' value is comparing variance across multiple runs before trusting a result.
- **Assuming no errors in the logs means the system is behaving correctly** — CoverageRecommendationAssessment (Batch 4) can execute successfully, with no error, and still

return the wrong recommendationTier; observability and evaluation answer two different questions, and you need both.

- **Bolting on evaluation and observability at the end of a project** — both are easiest to set up while you still remember what FraudRiskAssessment (Batch 1) was supposed to do; treat them as part of this week's build, not an afterthought before go-live.

17. Glossary

Term	Meaning
LLMOps	The operational discipline for LLM-powered systems — covering build, evaluate, observe/operate, and govern/release, built around prompts as a first-class artifact and non-determinism as an expected condition
Prompt Versioning	Treating a system or task prompt with the same version-control discipline as code: labeled versions, a change history, and a rollback path
Evaluation Suite	The overarching AIP Evals collection combining Test Cases, Target Function(s), and Evaluation Function(s)
Target Function	The function being evaluated by an Evaluation Suite — an AIP Logic function, a chatbot function, or a code-authored function
Evaluation Function	The method for comparing a Target Function's actual output against a Test Case's expected output
Test Case	A defined input and expected output, passed into the Evaluation Function(s) during a Suite run
Metric	A quantifiable, per-test-case result from a Suite run, used for aggregate scoring and for comparing variance across multiple runs
Foundry Observability	Foundry's monitoring, debugging, and analysis capabilities for live Functions, Actions, and AIP Logic executions
Monitoring	Rule-based, continuous checks on data health and execution behavior, with alerts routed to tools such as PagerDuty or Slack
Debugging	Tracing a specific execution's Workflow Lineage, logs, and operation-level detail to understand exactly what happened
Tracing	Drilling into a single operation within an execution to see its duration, inputs, outputs, and errors
Analysis	Exporting logs, metrics, and traces to a streaming dataset for custom, longer-horizon analysis workflows

Reflection Questions

- Pick one function or chatbot you've built in this program. What would its Evaluation Suite's Test Cases need to cover to catch a realistic failure mode?
- Where in your own work have you edited a prompt without recording what it said beforehand? What would prompt versioning have changed about that moment?
- For the same function or chatbot, which Foundry Observability capability — Monitoring, Debugging, or Analysis — would you reach for first if a policyholder reported an unexpected answer? Why that one first?
- AIP Evals' Metrics are only trustworthy when compared across multiple runs. What would “enough runs” look like for a function you'd be comfortable releasing on?

Further Practice

Continue into Day 2, where today's foundations extend into drift monitoring, cost management, release strategy, capacity planning, and AI governance — the bridge into Day 3's self-paced build of your own Production Readiness Package.